

네트워크 관련 라이브러리 Retrofit 과 Glide

통신 담당, Parsing 수행

이미지를 빠르게 다운받을 수 있다



목차

Retrofit 개요

Retrofit 구성

Retrofit 구현

- ◆필요 라이브러리 및 권한 추가
- ◆인터페이스 선언
- ◆POJO의 생성
- ◆실행코드 구현

Glide 개요

Retrofit 개요

☐ HTTP 클라이언트 서비스 라이브러리

☐ <https://square.github.io/retrofit/>

◆ 소스코드: <https://github.com/square/retrofit>

☐ REST 형식의 HTTP 요청처리

◆ GET/POST/UPDATE/DELETE
R C U D

☐ 응답 결과(XML or JSON)를 DTO로 변환 생성 가능

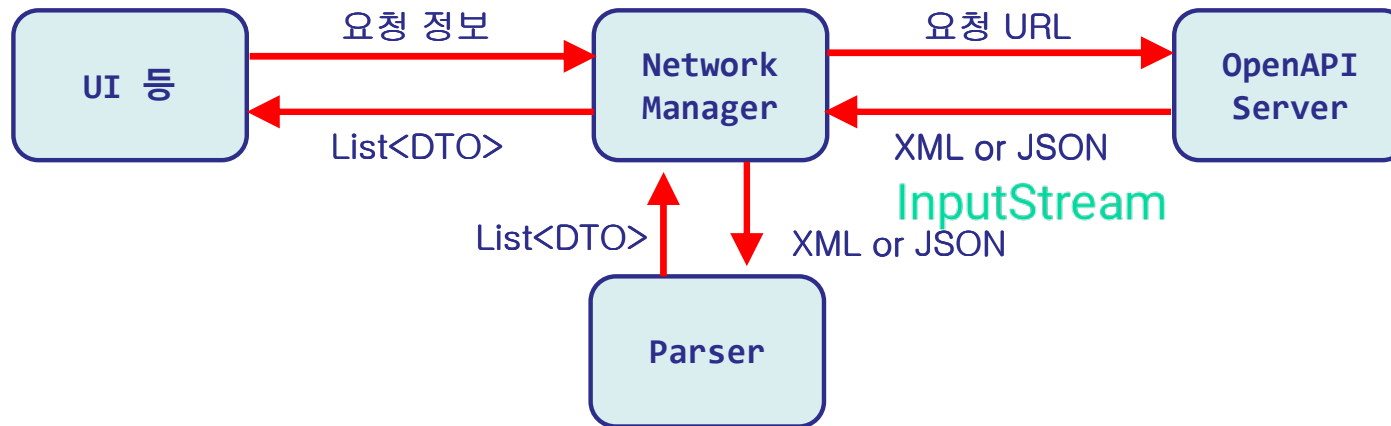
◆ OpenAPI 서버와의 통신 및 결과로 제공하는 XML(JSON) 결과 생성을 담당

◆ XML 또는 JSON 용 Converter 사용

■ JSON: GsonConverter

HTTP 요청 방식의 구성

HTTP 요청 방식 구조의 예



Retrofit 을 활용한 구조의 예



사용 준비

📖 Java 8 또는 Android API 21 이상 필요

📖 AndroidManifest.xml 에 네트워크 권한 추가

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
```

네트워크 사용 시 안드로이드 운영체제에 허락을 받아야 한다

📖 build.gradle (Module:app) 에 추가

◆ 현재 최신 버전: 2.11.0

```
dependencies {
    ...
    // retrofit
    val retrofit_version = "2.11.0"
    implementation("com.squareup.retrofit2:retrofit:$retrofit_version")
    implementation("com.squareup.retrofit2:converter-gson:$retrofit_version")
}
```

• Retrofit 기본 라이브러리

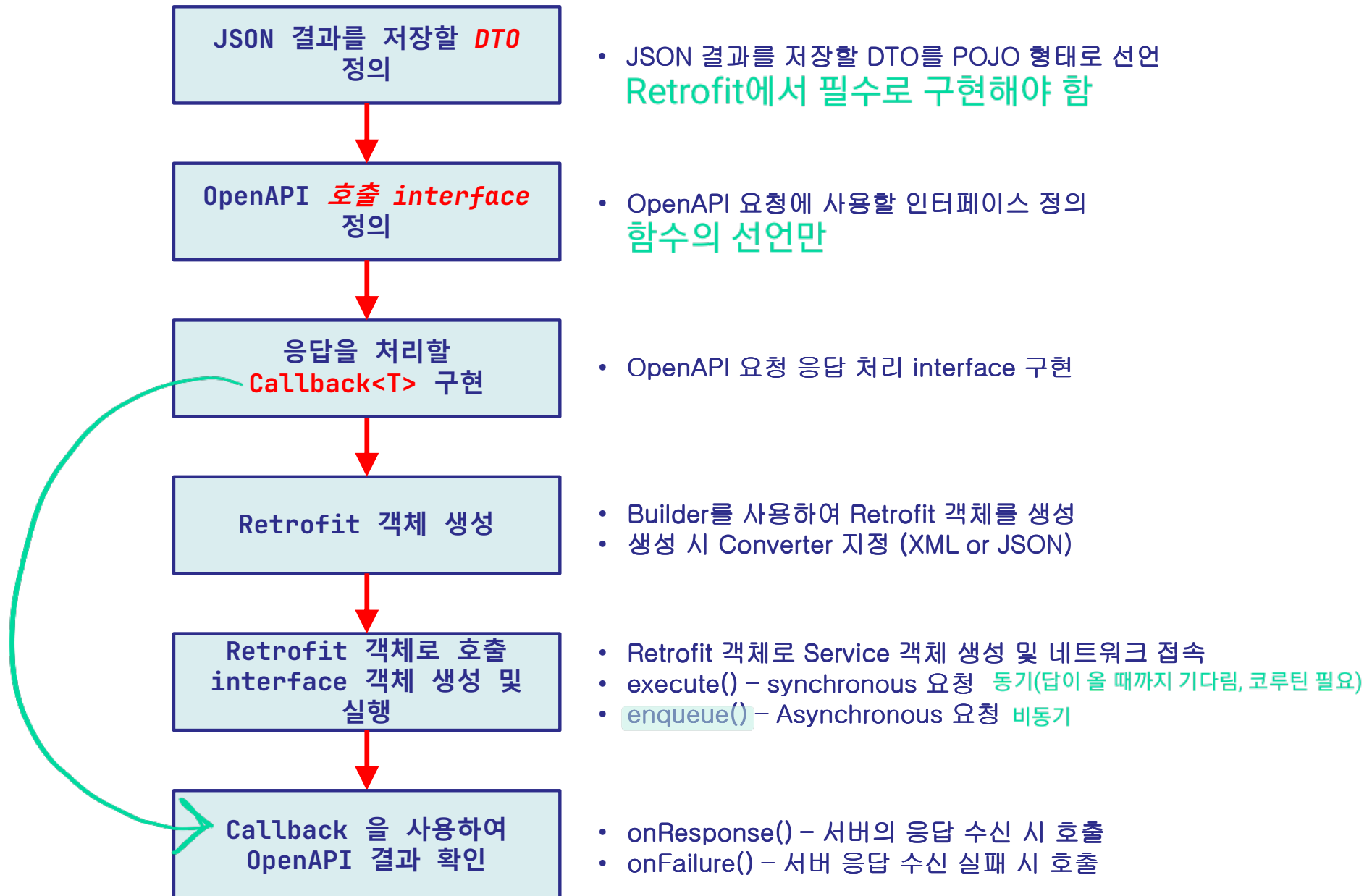
• gson: JSON 응답을 객체로 변환하기 위한 Converter

◆ XML의 경우

```
implementation("com.squareup.retrofit2:converter-simplexml:2.7.1")
```

Retrofit 사용 절차

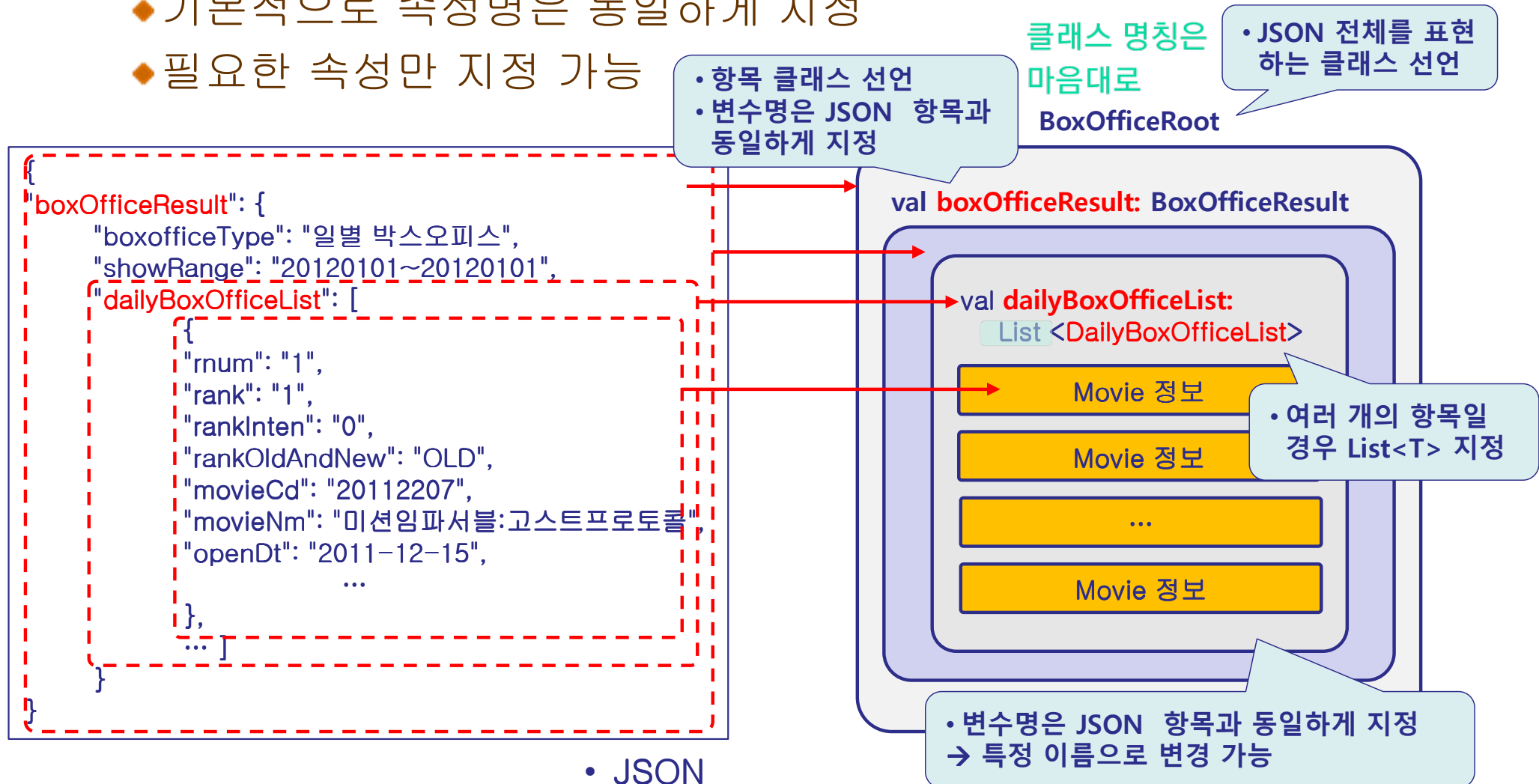
※ 이탤릭체로 표시한 부분은 사용자가 직접 구현



DTO 생성 01

결과를 저장할 DTO 선언 key값은 변수로 선언하되, 값으로 끝나지 않으면 클래스 형태로

- ◆ Json 의 구조를 Class 구조로 변경
- ◆ 기본적으로 속성명은 동일하게 지정
- ◆ 필요한 속성만 지정 가능



DTO 생성 02

 OpenAPI 서버에서 응답하는 XML 또는 JSON 구조로 data class 를 사용한 DTO 생성

- ◆ 필요 시 웹 상의 서비스 이용 가능
 - <https://transform.tools/json-to-kotlin> 등
- ◆ XML 또는 JSON 이 계층 구조를 갖고 있을 경우 각 계층 별로 클래스 생성
- ◆ 각 클래스를 개별 파일로 구현하거나 하나의 파일에 구현

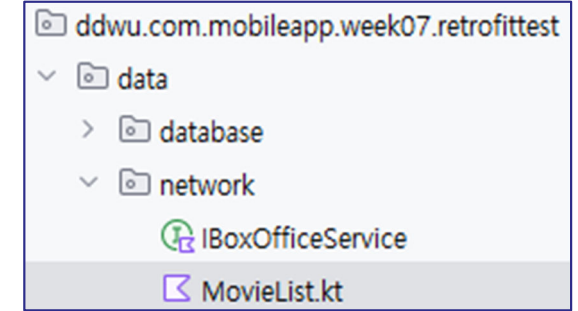
 패키지 및 클래스 추가

- ◆ Package 명: data
- ◆ BoxOfficeRoot: JSON 전체 표현
- ◆ BoxOfficeResult: 하위 항목
- ◆ DailyBoxOffice: 대상 항목

DTO 생성 03

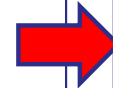
JSON → Data Class

• 최상위 시작 부분을 나타내는 클래스 → 클래스명 임의 지정



```
{
  "boxOfficeResult": {
    "boxofficeType": "일별 박스오피스",
    "showRange": "20120101~20120101",
    "dailyBoxOfficeList": [
      {
        "rnum": "1",
        "rank": "1",
        "rankInten": "0",
        "rankOldAndNew": "OLD",
        "movieCd": "20112207",
        "movieNm": "미션임파서블:고스트프로토콜",
        "openDt": "2011-12-15",
        ...
      },
      ...
    ]
  }
}
```

- JSON Object { ... }
- JSON Array [...]
- JSON Data key : value



```
data class Root(
    val boxOfficeResult: BoxOfficeResult,
)

data class BoxOfficeResult(
    val boxofficeType: String,
    val showRange: String,
    val dailyBoxOfficeList
        : List<DailyBoxOfficeList>,
) 항목이 있어서 리스트 형태로 클래스 선언

data class DailyBoxOfficeList(
    val rnum: String,
    val rank: String,
    val rankInten: String,
    val rankOldAndNew: String,
    val movieCd: String,
    val movieNm: String,
    val openDt: String,
    ...
)
```

• 불필요한 속성은 생략 가능

• JSON 배열의 항목을 표현하는 클래스

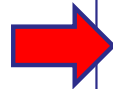
• JSON Data 를 표현하는 멤버변수

📁 항목의 이름 변경

`@SerializedName("JSON_NAME")`

`val NEW_NAME : TYPE` // JSON 에 나타난 항목 명을 다른 이름으로 바꾸고자 할 때

```
data class Root(  
    val boxOfficeResult: BoxOfficeResult,  
)  
  
data class BoxOfficeResult(  
    val boxofficeType: String,  
    val showRange: String,  
    val dailyBoxOfficeList  
        : List<DailyBoxOfficeList>,  
)  
  
data class DailyBoxOfficeList(  
    val rnum: String,  
    val rank: String,  
    val rankInten: String,  
    val rankOldAndNew: String,  
    val movieCd: String,  
    val movieNm: String,  
    val openDt: String,  
    ...  
)
```



```
data class Root(  
    @SerializedName("boxOfficeResult")  
    val movieResult: Result,  
)  
  
data class Result(  
    @SerializedName("dailyBoxOfficeList")  
    val movies: List<Movie>  
)  
  
data class Movie (  
    var rank: Int?,  
    @SerializedName("movieNm")  
    var title: String?,  
    @SerializedName("openDt")  
    var openDate: String?,  
)
```

• 클래스명은 임의 지정 가능

• 불필요 항목은 삭제

OpenAPI 요청 인터페이스 정의 01

요청 URL → Path명, Header, Parameter 등을 매개변수로 지정하는 **서비스 요청 함수** 정의

- ◆ **Call<T>** 객체를 반환 (예: Call<Root> - Root: JSON 최상위 클래스)
- ◆ REST 요청의 종류에 따라 annotation 지정
 - @GET, @POST, @PUT, @DELETE, and @HEAD

예 // 영화진흥위원회 OpenAPI URL
`http://www.kobis.or.kr/.json?key=key_value&targetDt=target_date`

Retrofit에게 나누어서 알려줘야 함

매개변수로 선언

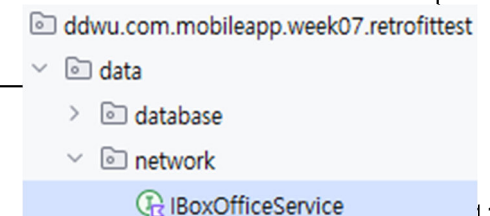
- **Domain:** `http://www.kobis.or.kr/` // strings.xml 에 지정 권장
- **Path:** `kobisopenapi/webservice/rest/boxoffice/searchDailyBoxOfficeList.json`
 // @GET(...) 등 Annotation 에서 사용
- **Query:** `key=key_value&targetDt=target_dt`

- @Path의 문자열을 메소드 매개변수로 대체
 - 하위경로: {변수명}
 - 매개변수: @Path("변수명") 변수 선언

```
interface IBoxOfficeService {
    @GET("kobisopenapi/webservice/rest/boxoffice/searchDailyBoxOfficeList.{type}")
    fun getDailyBoxOffice( @Path("type") type: String,
        @Query("key") key: String, @Query("targetDt") targetDate: String,
    ) : Call<Root>
}
```

- 반환 타입은 Call<T>
- T: JSON 최상위 표현 DTO
- → OpenAPI 호출을 표현

- OpenAPI 요청에 필요한 @Query 를 매개변수로 지정



OpenAPI 요청 인터페이스 정의 02

📖 관련 Annotation

@GET (“/path…/file_name” or “file_name”)

- REST 요청 중 GET 사용
- @POST, @PUT, @DELETE 등

@Path(“path_name”)

- URL의 경로 명을 지정할 때 사용
- URL의 {경로명} → @Path(“path_name”) 변수명 : 자료형

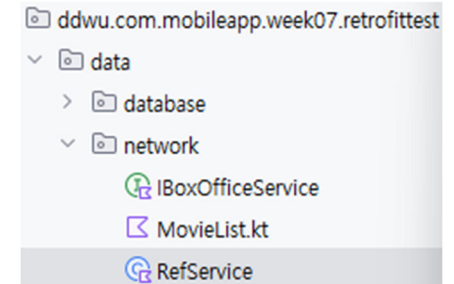
@Query(“parameter”)

- URL의 요청변수 명을 지정할 때 사용
- ?매개변수명=값& … → @Query(“path_name”) 변수명 : 자료형

@Header(“header_item”)

- Request 헤더에 저장할 변수 명을 지정할 때 사용
- 네이버 ClientID 와 ClientSecret 의 경우

```
fun getBooks (
    ...
    @Header("X-Naver-Client-Id") clientId: String,
    @Header("X-Naver-Client-Secret") clientSecret: String,
) : Call<Root>
```



Retrofit 실행 및 응답 처리 01

❏ Retrofit 객체 및 Service 객체 생성

- ◆ Retrofit 객체: Retrofit 서비스 제공
- ◆ Service 객체: OpenAPI 요청 인터페이스를 구현한 객체 → IBoxOfficeAPIService 를 구현한 객체

```
class RefService(val context: Context) {
    val TAG = "RefService"
    val movieService: IBoxOfficeService

    init {
        val retrofit : Retrofit = Retrofit.Builder()
            .baseUrl(context.resources.getString(R.string.kobis_url))
            .addConverterFactory( GsonConverterFactory.create() )
            .build()

        movieService = retrofit.create(IBoxOfficeService::class.java)
    }
    ...
}
```

• Retrofit 객체 생성

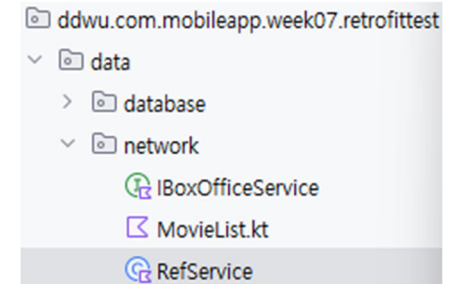
• OpenAPI 도메인 주소
• res/values/strings.xml 에 지정했을 경우 http://kobis.or.kr/

• JSON → DTO converter 설정

• Retrofit Service 객체
• Interface 를 구현한 서비스 객체 반환

• 서비스 요청 함수를 정의한 Interface 정보 전달
• getDailyBoxOffice()





Retrofit 실행 및 응답 처리 02

요청 함수 및 결과 수신 객체 생성

◆ Call → Callback<T> : T는 JSON ROOT 클래스

```
fun getMovies(key: String, date: String) : List<Movie>? {
```

• Movie 는 결과 DTO

```
    val movieCallback = object : Callback<Root> {
        override fun onResponse(call: Call<Root>, response: Response<Root>) {
            if (response.isSuccessful) {
                val boxOfficeRoot = response.body()
                val movies = boxOfficeRoot?.movieResult?.movieList
                movies?.forEach { movie ->
                    Log.d(TAG, movie.toString())
                }
            }
        }
    } // response 성공 시
```

• Root 는 최상위 DTO

• Callback<Root> 객체

```
    override fun onFailure(call: Call<Root>, t: Throwable) {
        Log.d(TAG, t.stackTraceToString())
    } // response 실패 시
```

• Retrofit 객체

```
    val movieCall = movieService.getDailyBoxOffice("json", key, date)
    movieCall.enqueue(movieCallback) // val response = movieCall.execute()
```

• 비동기 요청 실행

• 동기 요청 실행

```
    return null // return response.body()?.boxOfficeResult?.boxOfficeList
```

```
} // RefService 클래스
```

Retrofit 에서 Coroutine 사용

❏ Retrofit 은 Coroutine 을 통해 자체 비동기 처리 지원

◆ Call 및 Callback 사용 대체

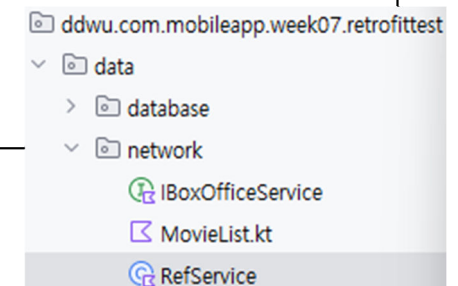
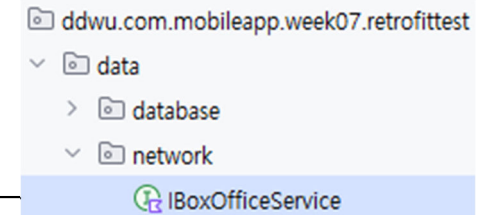
❏ 수정 사항

```
interface IBoxOfficeService {
    @GET("kobisopenapi/websevice/rest/boxoffice/searchDailyBoxOfficeList.{type}")
    suspend fun getDailyBoxOffice(           // suspend 함수로 변경

        ...

    ) : Root    // Call 사용 없이 JSON 표현 클래스 Root 반환
}
```

```
// suspend 함수로 변경
suspend fun getMovies(key: String, date: String) : List<Movie>? {
    // Call<Root> 가 아닌 Root 를 반환
    val root : ROOT = movieService.getDailyBoxOffice("json", key, date)
    // Root 에서 결과 추출
    return root.movieResult.movies // List<Movie>
}
```



네이버 검색 API 의 결과를 JSON 으로 받아 출력하는 앱을 구현

- ◆ NaverRetrofitSample.zip 사용
- ◆ 12페이지 헤더 추가 참고 (@Header)
- ◆ 네이버 책 검색 JSON 구조

```
{
  "lastBuildDate": "Tue, 11 Oct 2022 20:16:22 +0900",
  "total": 394,
  "start": 1,
  "display": 10,
  "items": [
    {
      "title": "Android",
      "link": "https://search.shopping.naver.com/book/catalogW/32451971823",
      "image": "https://shopping-phinf.pstatic.net/main_3245197W/32451971823.20220519081242.jpg",
      "author": "",
      "discount": "105270",
      "publisher": "Edizioni Accademiche Italiane",
      "pubdate": "20160114",
      "isbn": "9783639777833",
      "description": "Un metodo semplice per consentire a tutti gli appassionati di conoscere il mondo Android e sviluppare applicazioni per smartphone e tablet con semplicità. Riassunto in poche pagine tutto quello che c'è da sapere sul funzionamento delle App basate sul sistema operativo di casa Google. Seguendo questa guida, con semplici passaggi, potete creare tutte le applicazioni su misura per voi. All'interno troverete un'applicazione completa e funzionante d'esempio."
    }
  ]
}
```

• BookRoot 구현
-- items

• Item (Book) 구현
-- title
-- image
-- author
-- publisher

Glide 개요

Android 를 위한 Image loading library

- ◆ <https://bumptech.github.io/glide/>
- ◆ <https://github.com/bumptech/glide>
- ◆ 이미지의 빠른 로딩을 통한 부드러운 스크롤 제공
- ◆ 이미지 파일 저장 지원

사용준비

- ◆ Build.gradle(Module: ...)

```
dependencies {  
    // Glide  
    implementation 'com.github.bumptech.glide:glide:4.16.0'
```

Glide 사용 1

기본 구조

```
val url = resources.getString(R.string.image_url)

Glide.with(this)
    .load(url)
    .into(mainBinding.imageView)
```

ImageView

Glide 메소드

- ◆ with(...) : Context 정보 설정
- ◆ load(image) : 로딩할 이미지 정보, Bitmap, Drawable, String, URI 등
- ◆ into (ImageView) : 로딩 이미지를 보여줄 ImageView 객체
- ◆ placeholder (image) : 이미지 로딩 전 보여줄 이미지 지정
- ◆ error (image) : 로딩 에러 시 보여줄 이미지 지정
- ◆ fallback (image) : url 이 null 일 경우 보여줄 이미지 지정

Glide 사용 2

Bitmap 반환 구조

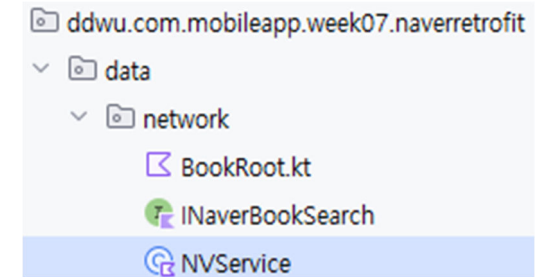
```
suspend fun getImage(url: String?) : Bitmap {
    val futureTarget : FutureTarget<Bitmap> =
        Glide.with(context)
            .asBitmap()
            .load(url)
            .submit()

    val bitmap = futureTarget.get()
    Glide.with(context).clear(futureTarget)

    return bitmap
}
```

• FutureTarget<Bitmap>
반환 형태로 변경
• CustomTarget<T>

• get() 을 사용하여 결과 확인



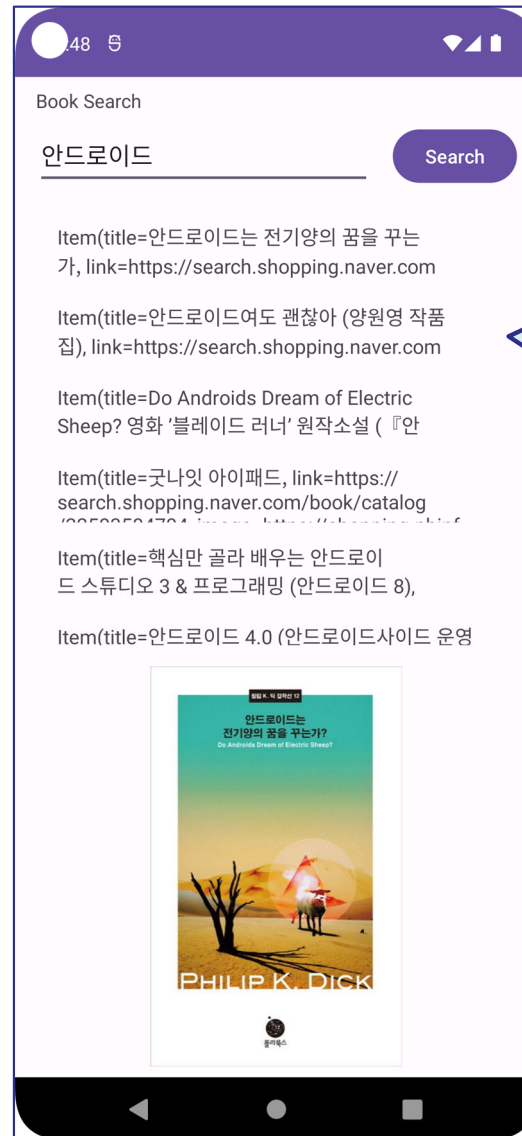
ImageView 에 전달

◆ ImageView.setImageBitmap(Bitmap) 사용

```
binding.imageView.setImageBitmap(bitmap)
```

실습 2

실습1을 RecyclerView 항목 클릭 시 해당 항목의 이미지 정보를 사용하여 Image를 표시



- 항목 클릭 시 Item 의 image 에 기록한 이미지주소를 Glide 에 전달하여 이미지 표시
- 구현해 놓은 `adapter.setOnItemClickListener ( )` 에 `BookAdapter.OnItemClickListener` 전달

